

Table of Contents

1. 前言	
2. 线程池核心设计与实现	
2.1 总体设计	
2.2 生命周期管理	
2.3 任务执行机制	
2.3.1 任务调度	
2.3.2 任务缓冲	
2.3.3 任务申请	
2.3.4 任务拒绝	
2.4 Worker线程管理	
2.4.1 Worker线程	
2.4.2 Worker线程增加	
2.4.4 Worker线程执行任务	
3 动态化线程池	
4. 使用	

1. 前言

线程池解决的核心问题就是资源管理问题。在并发环境下，系统不能够确定在任意时刻中，有多少任务需要执行，有多少资源需要投入。

为解决资源分配这个问题，线程池采用了“池化”（Pooling）思想。池化，顾名思义，是为了最大化收益并最小化风险，而将资源统一在一起管理的一种思想。

“池化”思想不仅仅能应用在计算机领域，在金融、设备、人员管理、工作管理等领域也有相关的应用。

在计算机领域中的表现为：统一管理IT资源，包括服务器、存储、和网络资源等等。通过共享资源，使用户在低投入中获益。除去线程池，还有其他比较典型的几种使用策略包括：

1. 内存池(Memory Pooling)：预先申请内存，提升申请内存速度，减少内存碎片。
2. 连接池(Connection Pooling)：预先申请数据库连接，提升申请连接的速度，降低系统的开销。
3. 实例池(Object Pooling)：循环使用对象，减少资源在初始化和释放时的昂贵损耗。

jdk 提供了快捷创建线程池的方式

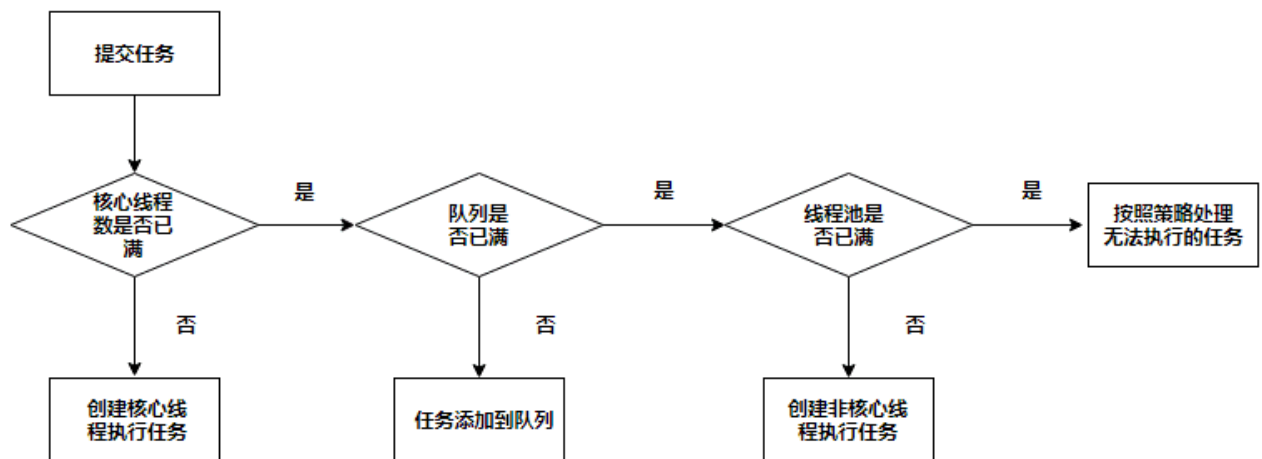
1. Executors 类下提供了定长/延时/单一等线程池,但由于上线使用系统资源,一般不建议使用

2. 手动创建 `ThreadPoolExecutor` 对象

其核心参数:

```
/**
 *
 * corePoolSize - 要保留在池中的线程数，即使它们处于空闲状态，除非设置了
 * allowCoreThreadTimeOut
 * maximumPoolSize - 池中允许的最大线程数
 * keepAliveTime - 当线程数大于核心数时，这是多余空闲线程在终止前等待新任务的最长时间。
 * unit - keepAliveTime参数的时间单位
 * workQueue - 用于在执行任务之前保存任务的队列。 这个队列将只保存execute方法提交的
 * Runnable任务
 */
public ThreadPoolExecutor(int corePoolSize,
                           int maximumPoolSize,
                           long keepAliveTime,
                           TimeUnit unit,
                           BlockingQueue<Runnable> workQueue) {
    this(corePoolSize, maximumPoolSize, keepAliveTime, unit, workQueue,
        Executors.defaultThreadFactory(), defaultHandler);
}
```

其执行流程



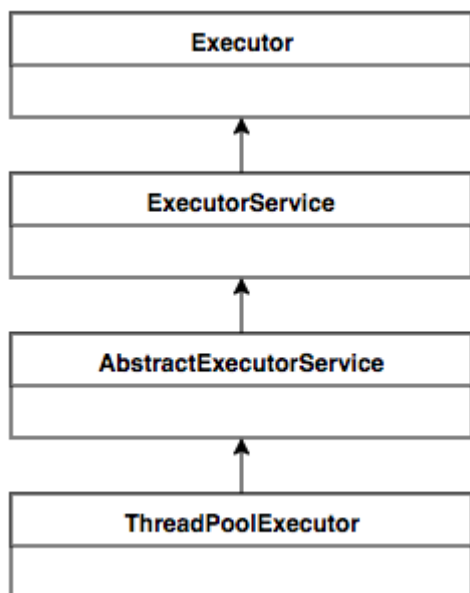
<https://blog.csdn.net/damokelisijian866>

为什么一般不建议使用Executors创建线程池? _damokelisijian866的博客-CSDN博客

2、线程池核心设计与实现

2.1 总体设计

Java中的线程池核心实现类是ThreadPoolExecutor，本章基于JDK 1.8的源码来分析Java线程池的核心设计与实现。我们首先来看一下ThreadPoolExecutor的UML类图，了解下ThreadPoolExecutor的继承关系。



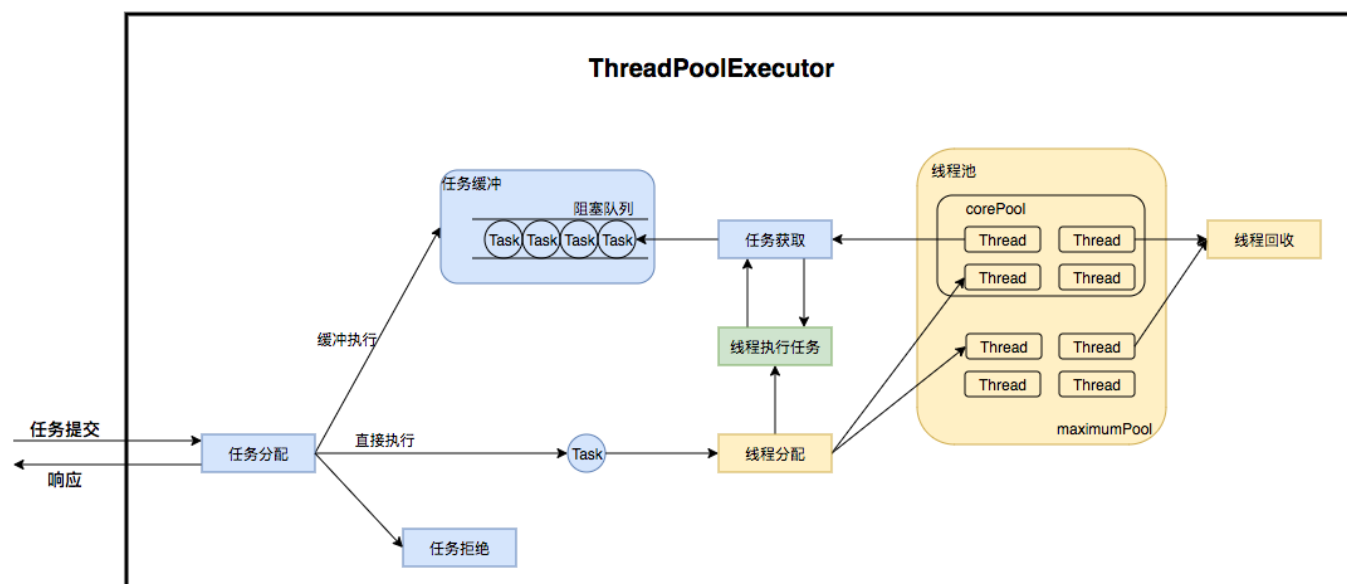
ThreadPoolExecutor实现的顶层接口是**Executor**，顶层接口Executor提供了一种思想：将任务提交和任务执行进行解耦。用户无需关注如何创建线程，如何调度线程来执行任务，用户只需提供Runnable对象，将任务的运行逻辑提交到执行器(Executor)中，由Executor框架完成线程的调配和任务的执行部分。**ExecutorService**接口增加了一些能力：

- (1) 扩充执行任务的能力，补充可以为一个或一批异步任务生成Future的方法；
- (2) 提供了管控线程池的方法，比如停止线程池的运行。

AbstractExecutorService则是上层的抽象类，将执行任务的流程串联了起来，保证下层的实现只需关注一个执行任务的方法即可。

最下层的实现类**ThreadPoolExecutor**实现最复杂的运行部分，ThreadPoolExecutor将会一方面维护自身的生命周期，另一方面同时管理线程和任务，使两者良好的结合从而执行并行任务。

ThreadPoolExecutor是如何运行，如何同时维护线程和执行任务的呢？其运行机制如下图所示：



线程池在内部实际上构建了一个生产者消费者模型，将线程和任务两者解耦，并不直接关联，从而良好的缓冲任务，复用线程。

线程池的运行主要分成两部分：任务管理、线程管理。

任务管理部分充当生产者的角色，当任务提交后，线程池会判断该任务后续的流转：

- 直接申请线程执行该任务；
- 缓冲到队列中等待线程执行；
- 拒绝该任务。

线程管理部分是消费者，它们被统一维护在线程池内，根据任务请求进行线程的分配，当线程执行完任务后则会继续获取新的任务去执行，最终当线程获取不到任务的时候，线程就会被回收。

2.2 生命周期管理

线程池运行的状态，并不是用户显式设置的，而是伴随着线程池的运行，由内部来维护。线程池内部使用一个变量维护两个值：运行状态(runState)和线程数量(workerCount)。在具体实现中，线程池将运行状态(runState)、线程数量(workerCount)两个关键参数的维护放在了一起，如下代码所示：

```
private final AtomicInteger ctl = new AtomicInteger(ctlOf(RUNNING, 0));
```

ctl 这个AtomicInteger类型，是对线程池的运行状态和线程池中有效线程的数量进行控制的一个字段，它同时包含两部分的信息：线程池的运行状态(runState)和线程池内有效线程的数量(workerCount)，高3位保存runState，低29位保存workerCount，两个变量之间互不干扰。用一个

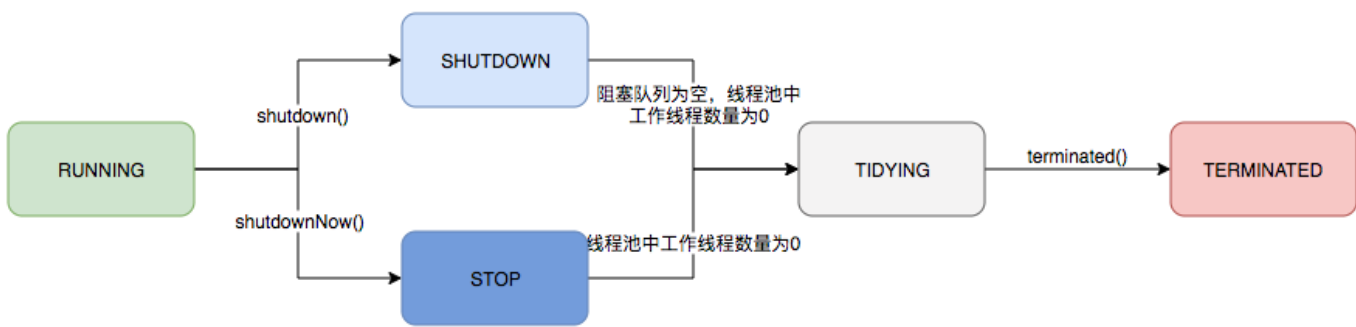
变量去存储两个值，可避免在做相关决策时，出现不一致的情况，不必为了维护两者的一致，而占用锁资源。通过阅读线程池源代码也可以发现，经常出现要同时判断线程池运行状态和线程数量的情况。线程池也提供了若干方法去供用户获得线程池当前的运行状态、线程个数。这里都使用的是位运算的方式，相比于基本运算，速度也会快很多。

关于内部封装的获取生命周期状态、获取线程池线程数量的计算方法如以下代码所示：

```
private static int runStateOf(int c)      { return c & ~CAPACITY; } //计算当前运行状态
private static int workerCountOf(int c)  { return c & CAPACITY; }  //计算当前线程数量
private static int ctlOf(int rs, int wc) { return rs | wc; }      //通过状态和线程数生成ctl
```

ThreadPoolExecutor的运行状态有5种，分别为：

运行状态	状态描述
RUNNING	能接受新提交的任务，并且也能处理阻塞队列中的任务。
SHUTDOWN	关闭状态，不再接受新提交的任务，但却可以继续处理阻塞队列中已保存的任务。
STOP	不能接受新任务，也不处理队列中的任务，会中断正在处理任务的线程。
TIDYING	所有的任务都已终止了，workerCount (有效线程数) 为0。
TERMINATED	在terminated() 方法执行完后进入该状态。



2.3 任务执行机制

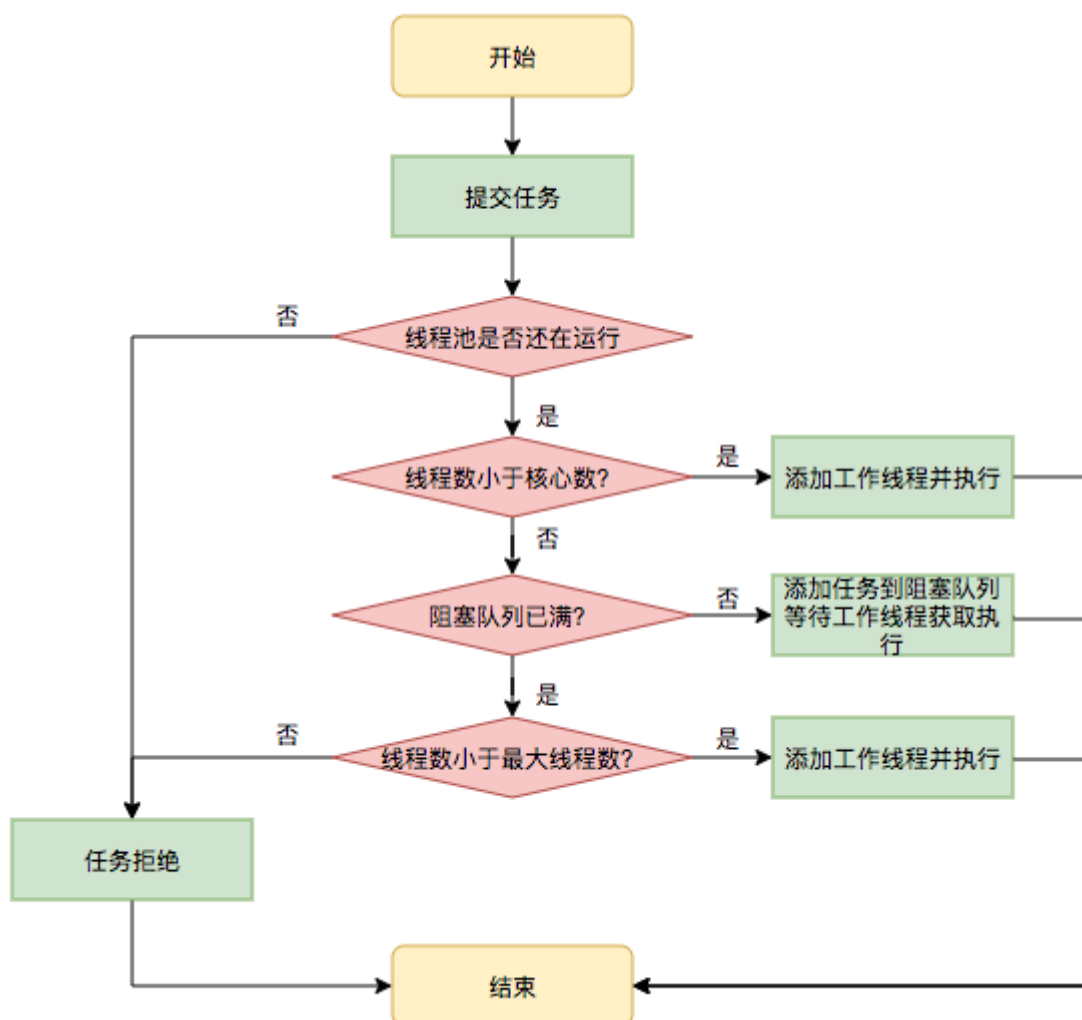
2.3.1 任务调度

任务调度是线程池的主要入口，当用户提交了一个任务，接下来这个任务将如何执行都是由这个阶段决定的。了解这部分就相当于了解了线程池的核心运行机制。

首先，所有任务的调度都是由execute方法完成的，这部分完成的工作是：检查现在线程池的运行状态、运行线程数、运行策略，决定接下来执行的流程，是直接申请线程执行，或是缓冲到队列中执行，亦或是直接拒绝该任务。其执行过程如下：

1. 首先检测线程池运行状态, 如果不是RUNNING, 则直接拒绝, 线程池要保证在RUNNING的状态下执行任务。
2. 如果 $workerCount < corePoolSize$, 则创建并启动一个线程来执行新提交的任务。
3. 如果 $workerCount \geq corePoolSize$, 且线程池内的阻塞队列未满, 则将任务添加到该阻塞队列中。
4. 如果 $workerCount \geq corePoolSize \ \&\& \ workerCount < maximumPoolSize$, 且线程池内的阻塞队列已满, 则创建并启动一个线程来执行新提交的任务。
5. 如果 $workerCount \geq maximumPoolSize$, 并且线程池内的阻塞队列已满, 则根据拒绝策略来处理该任务, 默认的处理方式是直接抛异常。

其执行流程如下图所示:



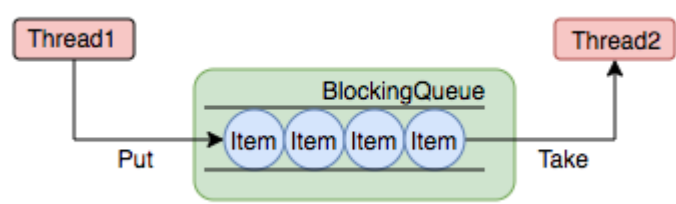
2.3.2 任务缓冲

任务缓冲模块是线程池能够管理任务的核心部分。线程池的本质是对任务和线程的管理, 而做到这一点最关键的思想就是将任务和线程两者解耦, 不让两者直接关联, 才可以做后续的分配工

作。线程池中是以生产者消费者模式，通过一个阻塞队列来实现的。阻塞队列缓存任务，工作线程从阻塞队列中获取任务。

阻塞队列(BlockingQueue)是一个支持两个附加操作的队列。这两个附加的操作是：在队列为空时，获取元素的线程会等待队列变为非空。当队列满时，存储元素的线程会等待队列可用。阻塞队列常用于生产者和消费者的场景，生产者是往队列里添加元素的线程，消费者是从队列里拿元素的线程。阻塞队列就是生产者存放元素的容器，而消费者也只从容器里拿元素。

下图中展示了线程1往阻塞队列中添加元素，而线程2从阻塞队列中移除元素：



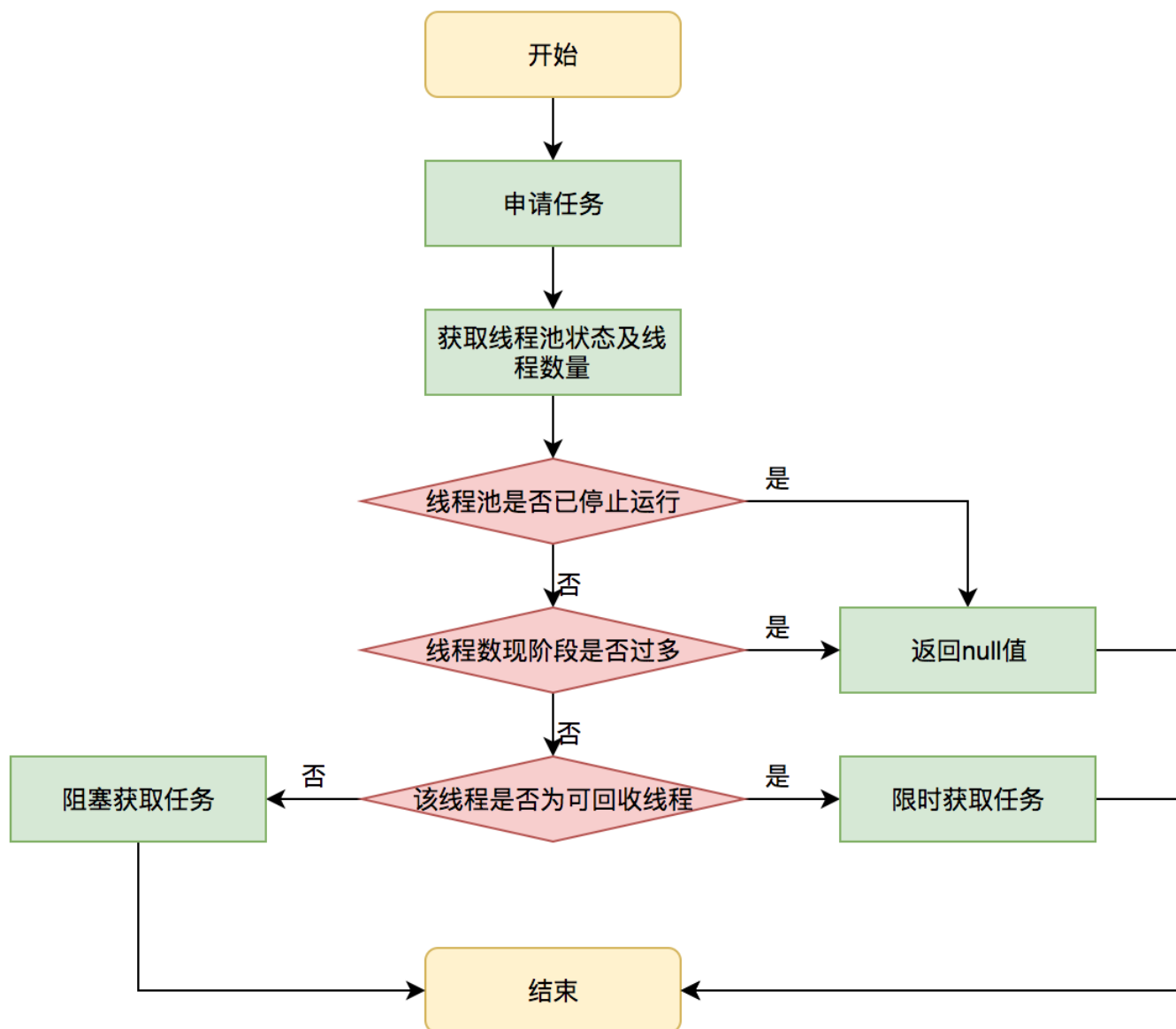
使用不同的队列可以实现不一样的任务存取策略。在这里，我们可以再介绍下阻塞队列的成员：

名称	描述
ArrayBlockingQueue	一个用数组实现的有界阻塞队列，此队列按照先进先出(FIFO)的原则对元素进行排序。支持公平锁和非公平锁。
LinkedBlockingQueue	一个由链表结构组成的有界队列，此队列按照先进先出(FIFO)的原则对元素进行排序。此队列的默认长度为Integer.MAX_VALUE，所以默认创建的该队列有容量危险。
PriorityBlockingQueue	一个支持线程优先级排序的无界队列，默认自然序进行排序，也可以自定义实现compareTo()方法来指定元素排序规则，不能保证同优先级元素的顺序。
DelayQueue	一个实现PriorityBlockingQueue实现延迟获取的无界队列，在创建元素时，可以指定多久才能从队列中获取当前元素。只有延时期满后才能从队列中获取元素。
SynchronousQueue	一个不存储元素的阻塞队列，每一个put操作必须等待take操作，否则不能添加元素。支持公平锁和非公平锁。SynchronousQueue的一个使用场景是在线程池里。Executors.newCachedThreadPool()就使用了SynchronousQueue，这个线程池根据需要（新任务到来时）创建新的线程，如果有空闲线程则会重复使用，线程空闲了60秒后会被回收。
LinkedTransferQueue	一个由链表结构组成的无界阻塞队列，相当于其它队列，LinkedTransferQueue队列多了transfer和tryTransfer方法。
LinkedBlockingDeque	一个由链表结构组成的双向阻塞队列。队列头部和尾部都可以添加和移除元素，多线程并发时，可以将锁的竞争最多降到一半。

2.3.3 任务申请

由上文的任务分配部分可知，任务的执行有两种可能：一种是任务直接由新创建的线程执行。另一种是线程从任务队列中获取任务然后执行，执行完任务的空闲线程会再次去从队列中申请任务再去执行。第一种情况仅出现在线程初始创建的时候，第二种是线程获取任务绝大多数的情况。

线程需要从任务缓存模块中不断地取任务执行，帮助线程从阻塞队列中获取任务，实现线程管理模块和任务管理模块之间的通信。这部分策略由getTask方法实现，其执行流程如下图所示：



```
java.util.concurrent.ThreadPoolExecutor#getTask
```

`getTask`这部分进行了多次判断，为的是控制线程的数量，使其符合线程池的状态。如果线程池现在不应该持有那么多线程，则会返回`null`值。工作线程`Worker`会不断接收新任务去执行，而当工作线程`Worker`接收不到任务的时候，就会开始被回收。

2.3.4 任务拒绝

任务拒绝模块是线程池的保护部分，线程池有一个最大的容量，当线程池的任务缓存队列已满，并且线程池中的线程数目达到`maximumPoolSize`时，就需要拒绝掉该任务，采取任务拒绝策略，保护线程池。

拒绝策略是一个接口，其设计如下：

```
public interface RejectedExecutionHandler {  
    void rejectedExecution(Runnable r, ThreadPoolExecutor executor);  
}
```

用户可以通过实现这个接口去定制拒绝策略，也可以选择JDK提供的四种已有拒绝策略，其特点如下：

	名称	描述
1	<code>ThreadPoolExecutor.AbortPolicy</code>	丢弃任务并抛出 <code>RejectedExecutionException</code> 异常。这是线程池默认的拒绝策略，在任务不能再提交的时候，抛出异常，及时反馈程序运行状态。如果是比较关键的业务，推荐使用此拒绝策略，这样子在系统不能承载更大的并发量的时候，能够及时的通过异常发现。
2	<code>ThreadPoolExecutor.DiscardPolicy</code>	丢弃任务，但是不抛出异常。使用此策略，可能会使我们无法发现系统的异常状态。建议是一些无关紧要的业务采用此策略。
3	<code>ThreadPoolExecutor.DiscardOldestPolicy</code>	丢弃队列最前面的任务，然后重新提交被拒绝的任务。是否要采用此种拒绝策略，还得根据实际业务是否允许丢弃老任务来认真衡量。
4	<code>ThreadPoolExecutor.CallerRunsPolicy</code>	由调用线程（提交任务的线程）处理该任务。这种情况是需要让所有任务都执行完毕，那么就适合大量计算的任务类型去执行，多线程仅仅是增大吞吐量的手段，最终必须要让每个任务都执行完毕。

2.4 Worker线程管理

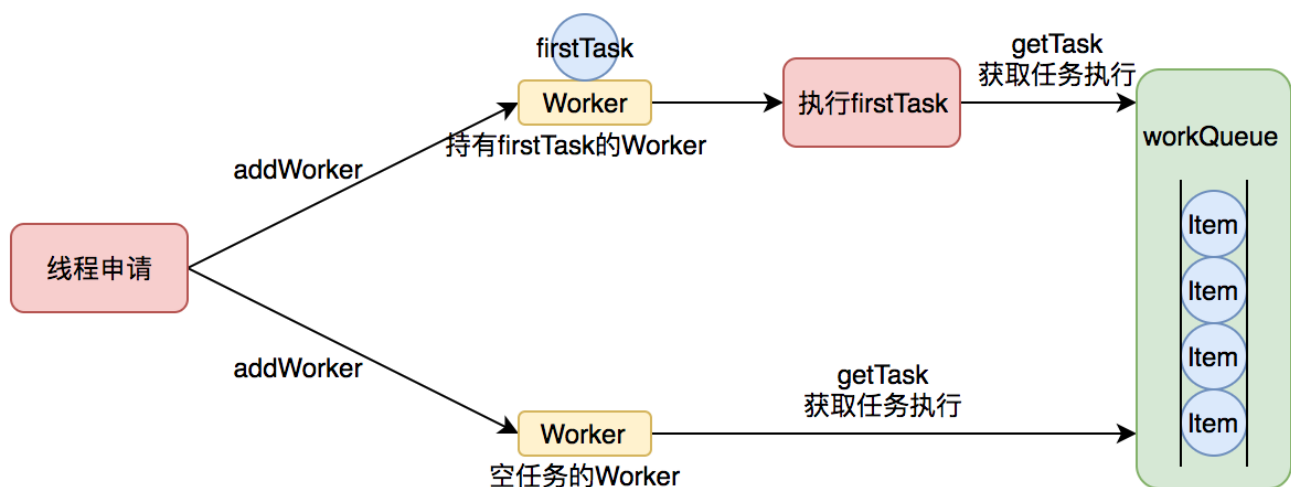
2.4.1 Worker线程

线程池为了掌握线程的状态并维护线程的生命周期，设计了线程池内的工作线程Worker。我们来看一下它的部分代码：

```
private final class Worker extends AbstractQueuedSynchronizer implements Runnable{  
    final Thread thread;//Worker持有的线程  
    Runnable firstTask;//初始化的任务，可以为null  
}
```

Worker这个工作线程，实现了Runnable接口，并持有一个线程thread，一个初始化的任务firstTask。thread是在调用构造方法时通过ThreadFactory来创建的线程，可以用来执行任务；firstTask用它来保存传入的第一个任务，这个任务可以有也可以为null。如果这个值是非空的，那么线程就会在启动初期立即执行这个任务，也就对应核心线程创建时的情况；如果这个值是null，那么就需要创建一个线程去执行任务列表（workQueue）中的任务，也就是非核心线程的创建。

Worker执行任务的模型如下图所示：

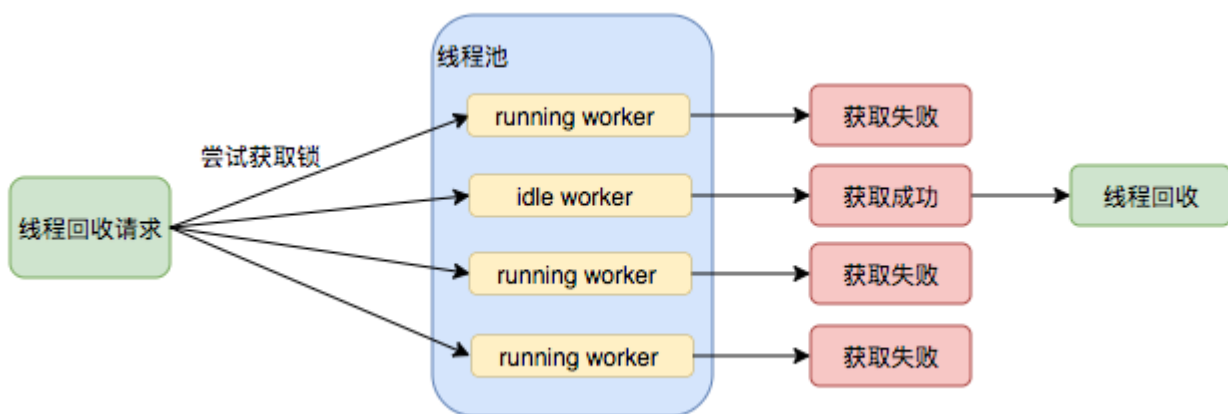


线程池需要管理线程的生命周期，需要在线程长时间不运行的时候进行回收。线程池使用一张Hash表去持有线程的引用，这样可以通过添加引用、移除引用这样的操作来控制线程的生命周期。这个时候重要的就是如何判断线程是否在运行。

Worker是通过继承AQS，使用AQS来实现独占锁这个功能。没有使用可重入锁ReentrantLock，而是使用AQS，为的就是实现不可重入的特性去反应线程现在的执行状态。

1.lock方法一旦获取了独占锁，表示当前线程正在执行任务中。 2.如果正在执行任务，则不应该中断线程。 3.如果该线程现在不是独占锁的状态，也就是空闲的状态，说明它没有在处理任务，这时可以对该线程进行中断。 4.线程池在执行shutdown方法或tryTerminate方法时会调用interruptIdleWorkers方法来中断空闲的线程，interruptIdleWorkers方法会使用tryLock方法来判断线程池中的线程是否是空闲状态；如果线程是空闲状态则可以安全回收。

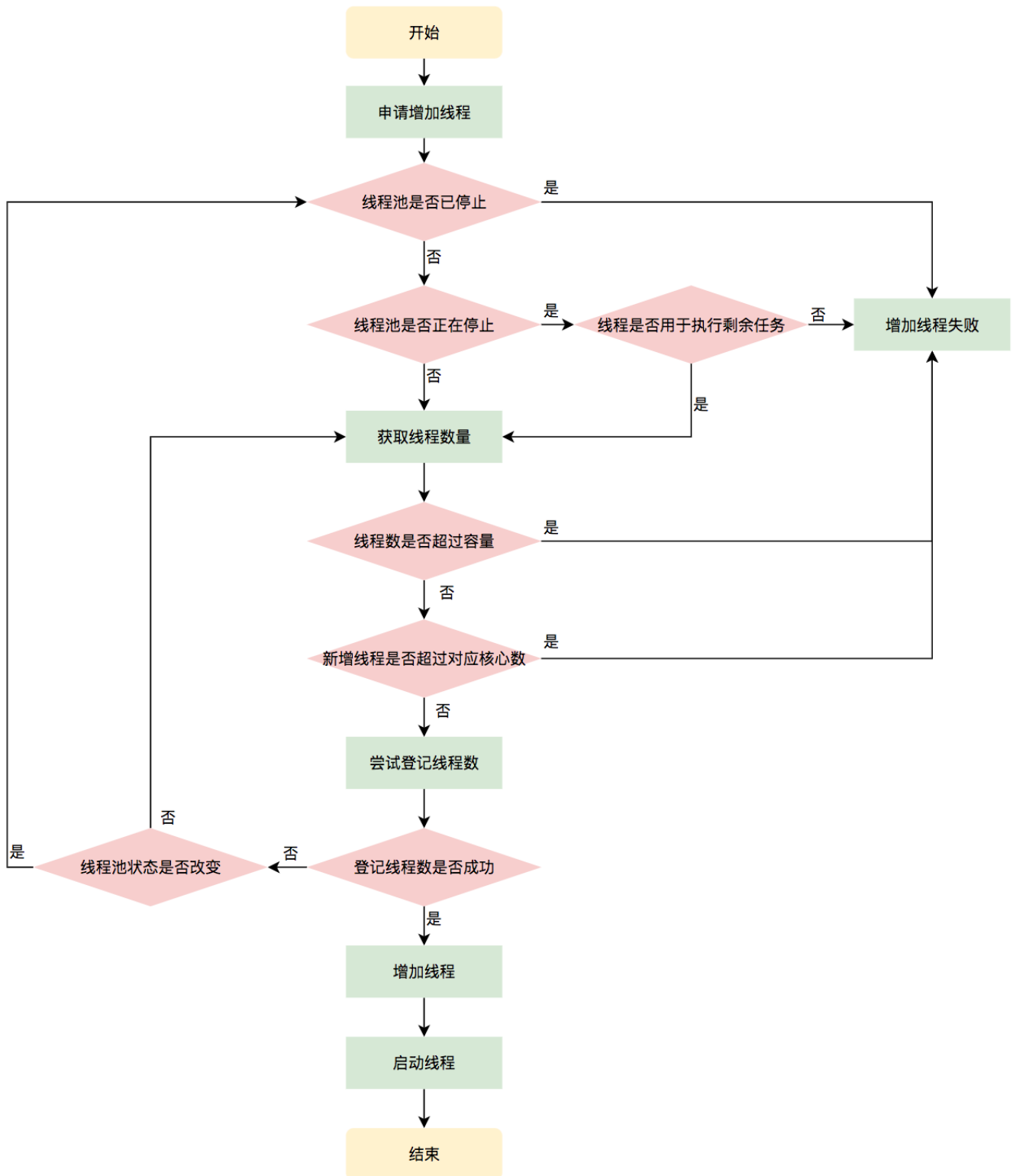
在线程回收过程中就使用到了这种特性，回收过程如下图所示：



2.4.2 Worker线程增加

增加线程是通过线程池中的addWorker方法，该方法的功能就是增加一个线程，该方法不考虑线程池是在哪个阶段增加的该线程，这个分配线程的策略是在上个步骤完成的，该步骤仅仅完成增加线程，并使它运行，最后返回是否成功这个结果。addWorker方法有两个参数：firstTask、

core。firstTask参数用于指定新增的线程执行的第一个任务，该参数可以为空；core参数为true表示在新增线程时会判断当前活动线程数是否少于corePoolSize，false表示新增线程前需要判断当前活动线程数是否少于maximumPoolSize，其执行流程如下图所示：



2.4.3 Worker线程回收

线程池中线程的销毁依赖JVM自动的回收，线程池做的工作是根据当前线程池的状态维护一定数量的线程引用，防止这部分线程被JVM回收，当线程池决定哪些线程需要回收时，只需要将其引用消除即可。Worker被创建出来后，就会不断地进行轮询，然后获取任务去执行，核心线程可以无限等待获取任务，非核心线程要限时获取任务。当Worker无法获取到任务，也就是获取的任务为空时，循环会结束，Worker会主动消除自身在线程池内的引用。

```
try {
    while (task != null || (task = getTask()) != null) {
        //执行任务
    }
} finally {
    processWorkerExit(w, completedAbruptly); //获取不到任务时，主动回收自己
}
```

线程回收的工作是在processWorkerExit方法完成的。

java.util.concurrent.ThreadPoolExecutor#processWorkerExit

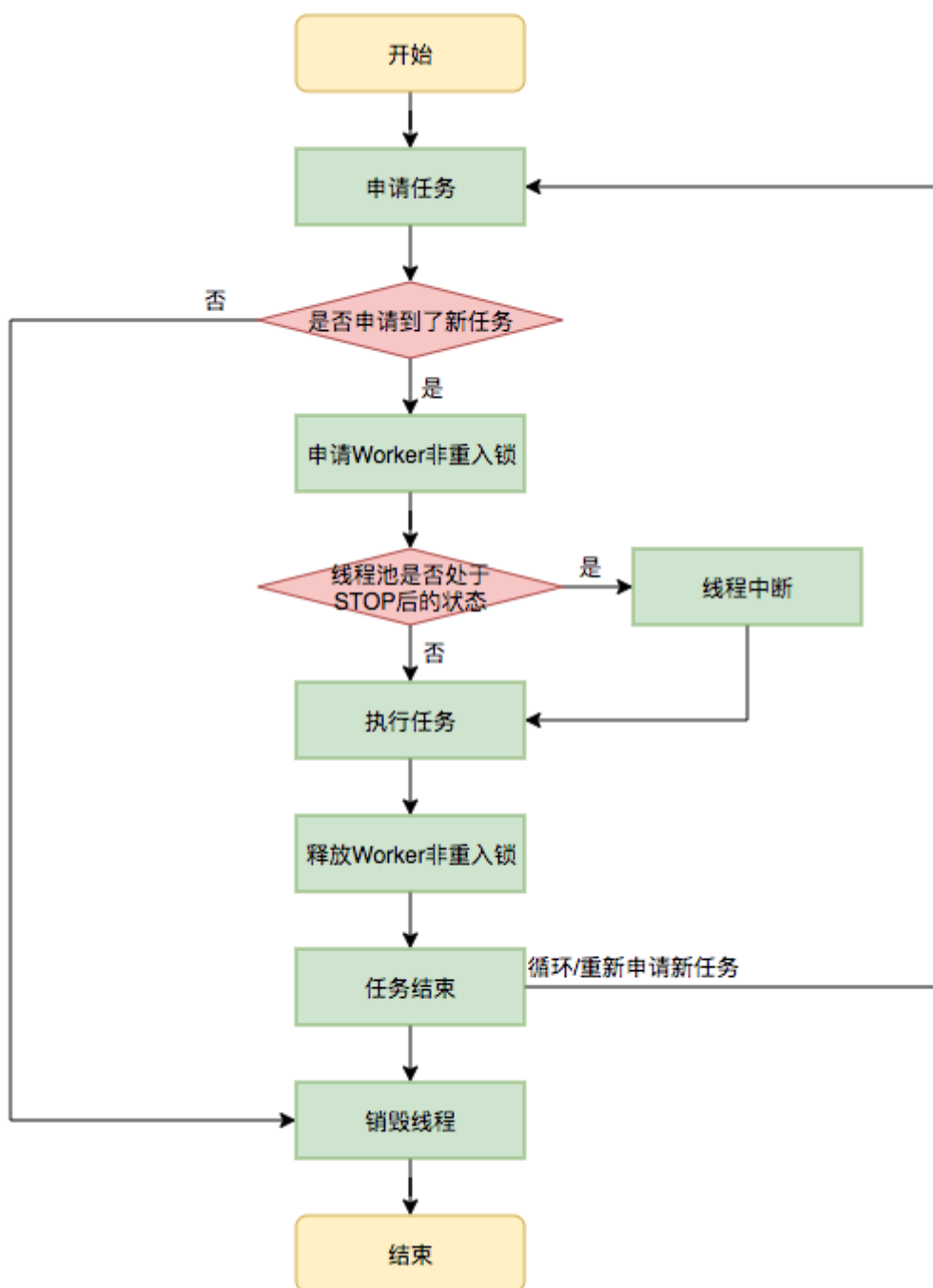


事实上，在这个方法中，将线程引用移出线程池就已经结束了线程销毁的部分。但由于引起线程销毁的可能性有很多，线程池还要判断是什么引发了这次销毁，是否要改变线程池的现阶段状态，是否要根据新状态，重新分配线程。

2.4.4 Worker线程执行任务

在Worker类中的run方法调用了runWorker方法来执行任务，runWorker方法的执行过程如下：

1. while循环不断地通过getTask()方法获取任务。
2. getTask()方法从阻塞队列中取任务。
3. 如果线程池正在停止，那么要保证当前线程是中断状态，否则要保证当前线程不是中断状态。
4. 执行任务。
5. 如果getTask结果为null则跳出循环，执行processWorkerExit()方法，销毁线程。



3 动态化线程池

线程池使用面临的核心的问题在于：**线程池的参数并不好配置**

不同场景不同业务不同时间段等 对线程池的使用不一样,所以需要动态参数, 可以借助配置中心等方式实现

还可以监控线程池的使用, **任务监控**, **负载告警** 等等

[Java线程池实现原理及其在美团业务中的实践 - 美团技术团队 \(meituan.com\)](#)

4. 使用

常用的几个阻塞队列：

1. `LinkedBlockingQueue`

链式阻塞队列，底层数据结构是链表，默认大小是 `Integer.MAX_VALUE`，也可以指定大小。

2. `ArrayBlockingQueue`

数组阻塞队列，底层数据结构是数组，需要指定队列的大小。

3. `SynchronousQueue`

同步队列，内部容量为0，每个put操作必须等待一个take操作，反之亦然。

4. `DelayQueue`

延迟队列，该队列中的元素只有当其指定的延迟时间到了，才能够从队列中获取到该元素。

bean 注入

```

package com.gree.ecommerce.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.Primary;
import org.springframework.scheduling.annotation.EnableAsync;
import org.springframework.scheduling.concurrent.ThreadPoolTaskExecutor;

import java.util.concurrent.RejectedExecutionHandler;
import java.util.concurrent.ThreadPoolExecutor;

@Configuration
@EnableAsync
public class ThreadPoolConfig {
    /**
     *
     */
    private static final int MULTIPLE = 5;
    /**
     *
     */
    private static final int SECKILL_MULTIPLE = 10000;
    /**
     *
     */
    private static final int ALIVE_TIMEOUT = 30;
    /**
     * 核心线程数：线程池创建时候初始化的线程数
     */
    private final int corePoolSize = Runtime.getRuntime().availableProcessors();

    /**
     * 最大线程数：线程池最大的线程数，只有在缓冲队列满了之后才会申请超过核心线程数的线程
     */
    private final int maxPoolSize = Runtime.getRuntime().availableProcessors() *
MULTIPLE;

    /**
     * 缓冲队列：用来缓冲执行任务的队列
     */
    private final int queueCapacity = maxPoolSize * MULTIPLE;

    /**
     * 缓冲队列：用来缓冲执行任务的队列
     */
    private final int seckillQueueCapacity = maxPoolSize * SECKILL_MULTIPLE;

    /**
     * 允许线程的空闲时间(单位：秒)：当超过了核心线程出之外的线程在空闲时间到达之后会被销毁

```

```

    */
    private final int keepAliveSeconds = ALIVE_TIMEOUT;

    /**
     * 线程池名的前缀：设置好了之后可以方便我们定位处理任务所在的线程池
     */
    private String seckillThreadNamePrefix = "seckill-";

    @Bean(name = "managePoolTaskExecutor")
    @Primary
    public ThreadPoolTaskExecutor getManagePoolTaskExecutor() {
        return generateThreadPoolTaskExecutor(corePoolSize, maxPoolSize,
            queueCapacity, keepAliveSeconds, threadNamePrefix, new
            ThreadPoolExecutor.CallerRunsPolicy());
    }

    /**
     * 秒杀线程池
     */
    @Bean(name = "seckillPoolTaskExecutor")
    public ThreadPoolTaskExecutor getSeckillPoolTaskExecutor() {
        return generateThreadPoolTaskExecutor(corePoolSize, maxPoolSize,
            seckillQueueCapacity, keepAliveSeconds, seckillThreadNamePrefix, new
            ThreadPoolExecutor.DiscardPolicy());
    }

    /**
     *
     * @param threadNamePrefix 异步方法内部线程名称
     * @param corePoolSize 核心线程数
     * @param maxPoolSize 线程池维护线程的最大数量,只有在缓冲队列满了之后才会申请超过核心
     线程数的线程
     * @param queueCapacity 缓存队列
     * @param keepAliveSeconds 允许的空闲时间,当超过了核心线程数之外的线程在空闲时间到达之
     后会被销毁
     * @return ThreadPoolTaskExecutor
     */
    private ThreadPoolTaskExecutor generateThreadPoolTaskExecutor(int corePoolSize,
        int maxPoolSize, int queueCapacity, int keepAliveSeconds, String threadNamePrefix,
        RejectedExecutionHandler rejectedExecutionHandler){
        ThreadPoolTaskExecutor taskExecutor = new ThreadPoolTaskExecutor();
        taskExecutor.setCorePoolSize(corePoolSize);
        taskExecutor.setMaxPoolSize(maxPoolSize);
        taskExecutor.setQueueCapacity(queueCapacity);
        taskExecutor.setKeepAliveSeconds(keepAliveSeconds);
        taskExecutor.setThreadNamePrefix(threadNamePrefix);
        taskExecutor.setRejectedExecutionHandler(rejectedExecutionHandler);
        //当调度器shutdown被调用时等待当前被调度的任务完成
        taskExecutor.setWaitForTasksToCompleteOnShutdown(true);
        taskExecutor.initialize();
    }

```

```
    return taskExecutor;  
}  
}
```